

Arquitectura final — Sistema Empresarial de Tickets, Chat, Inventario y Plantillas

1. Visión final del sistema

El sistema debe quedar como una plataforma empresarial **modular por departamentos**, con backend central en NestJS, aplicación web administrativa y operativa, y aplicación móvil nativa en Flutter independiente de la web. La arquitectura propuesta conserva tu enfoque original de tickets, chat, inventario y automatización, pero lo reorganiza para que cada departamento tenga su propio módulo, sus formularios, sus plantillas y su lógica operativa sin romper el núcleo común.

La idea clave del diseño final es esta:

- Núcleo común: autenticación, usuarios, permisos, tickets, chat, adjuntos, notificaciones, auditoría, webhooks e integraciones.
- Capa de negocio por departamento: Biomédico, Sistemas, Mantenimiento de Precisión, Aire Acondicionado, y futuros departamentos.
- Capa de plantillas configurable por base de datos: formularios, campos, flujos, estados, checklist, SLA, tipos de ticket y cierre por área.
- Canales cliente independientes: web empresarial y app móvil nativa Flutter.
- Infraestructura open source al máximo: PostgreSQL, Redis, MinIO, Nginx, Docker Compose, NestJS, Flutter, n8n, Keycloak opcional si luego quieres evolucionar autenticación centralizada.

2. Principios de arquitectura

2.1 Enfoque general

Se recomienda una arquitectura **modular monolítica** en la primera etapa, no microservicios desde el día uno.

¿Por qué?

- Para 1000 empleados, un monolito modular en NestJS soporta perfectamente el volumen esperado.
- Reduce complejidad operativa, despliegue, observabilidad y costos.
- Permite separar dominios internamente por módulos y luego extraer servicios si el crecimiento lo exige.
- Encaja mejor con EasyPanel, Docker Compose y despliegue rápido empresarial.

2.2 Patrón recomendado

- Backend: NestJS + PostgreSQL + Redis + MinIO.
- API principal: REST para operaciones administrativas y móviles.
- Tiempo real: WebSocket Gateway con Socket.IO.
- Jobs y automatizaciones: BullMQ con Redis.
- Integraciones: Webhooks firmados + API tokens por sistema/área.
- Búsqueda avanzada futura: OpenSearch o Elasticsearch en fase 2.

2.3 Objetivo estructural

El sistema debe permitir que un departamento nuevo se agregue **sin reescribir código core**, solo creando:

- su configuración de departamento,
- sus plantillas,
- sus tipos de activos,
- sus campos personalizados,
- sus reglas de asignación,
- sus flujos de aprobación o cierre.

3. Arquitectura funcional final

3.1 Canales de acceso

Web

Pensada para:

- admin
- supervisor
- mesa de ayuda
- auditoría
- técnicos cuando trabajen desde PC

App móvil nativa Flutter

Pensada para:

- técnico en campo
- supervisor móvil
- solicitante general

- auditor de solo lectura
- administrador móvil para monitoreo rápido

La app Flutter será **independiente** del frontend web, compartiendo únicamente API, autenticación, reglas de negocio y diseño de permisos.

4. Roles finales del sistema

Rol	Alcance principal	Web	Flutter
admin	Configuración total, seguridad, integraciones, plantillas, departamentos	Sí	Sí
supervisor	Gestión operativa por departamento, aprobación, asignación, métricas	Sí	Sí
tecnico_area	Ejecución, atención de tickets, cierre, chat, checklist, fotos	Sí	Sí
solicitante	Crear ticket, seguimiento, chat, adjuntos, historial propio	Sí	Sí
auditor	Lectura de reportes, exportaciones, trazabilidad, cumplimiento	Sí	Sí
integracion_api	Usuario técnico para n8n, bots, sistemas externos	No	No

5. Diseño por departamentos

Cada departamento debe existir como una entidad configurable en base de datos, no como código fijo duro.

5.1 Departamentos base iniciales

- Biomédico
- Sistemas
- Mantenimiento de Precisión
- Aire Acondicionado
- Infraestructura / Servicios Generales (opcional)
- Otros futuros

5.2 Qué define un departamento

Cada departamento tendrá:

- nombre, código y estado
- supervisor responsable
- técnicos asociados
- tipos de ticket habilitados
- plantillas de formulario disponibles
- tipos de activos permitidos

- reglas de SLA
- reglas de notificación
- flujo de estados permitido
- checklist de atención y cierre
- permisos especiales de visualización o edición

5.3 Plantillas desacopladas

Debe existir un módulo de plantillas independiente, para luego asociar plantillas a departamentos.

Eso te da estas ventajas:

- puedes reutilizar plantillas entre áreas,
- puedes versionar formularios,
- puedes activar o desactivar campos sin desplegar código,
- puedes cambiar flujo operativo por base de datos,
- puedes evolucionar a departamentos dinámicos en fase 3.

6. Módulos finales del backend en NestJS

La estructura propuesta en NestJS debe quedar separada por dominios:

```
apps/
  api/
    src/
      main.ts
      app.module.ts
      common/
        decorators/
        guards/
        interceptors/
        filters/
        pipes/
        dtos/
        enums/
        utils/
      config/
        env/
        database/
        redis/
        storage/
        ws/
      modules/
        auth/
        users/
        roles/
        departments/
```

```
templates/  
dynamic-fields/  
tickets/  
ticket-workflows/  
assignments/  
inventory/  
assets/  
maintenance-records/  
chat/  
notifications/  
attachments/  
comments/  
audit/  
webhooks/  
integrations/  
api-tokens/  
reports/  
dashboard/  
mobile/  
ai-classification/
```

6.1 Descripción de módulos

auth

- login por cédula
- generación y validación OTP
- sesión JWT + refresh token
- cierre de sesión
- control de dispositivos

users

- empleados
- perfil
- teléfonos
- áreas asociadas
- estados del usuario

roles

- RBAC
- permisos granulares
- perfiles por canal

departments

- catálogo de departamentos
- relación con supervisores, técnicos, activos y plantillas

templates

- plantillas de formularios
- plantillas de checklist
- plantillas de flujo
- versionado

dynamic-fields

- definición de campos dinámicos
- validación por tipo
- select, texto, fecha, boolean, archivo, serial, QR, firma

tickets

- creación, edición, seguimiento, cierre
- estados y transiciones
- prioridad, criticidad, categoría, subcategoría

ticket-workflows

- reglas de avance por estado
- reglas por departamento
- SLA y escalamiento

assignments

- asignación manual, automática, por supervisor o por reglas

inventory

- activos por departamento
- seriales, QR, etiquetas, fabricante, modelo, ubicación

assets

- hoja de vida del equipo
- estado operativo
- datos clínicos/técnicos según tipo

maintenance-records

- historial técnico
- mantenimientos preventivos/correctivos
- vínculo automático al cierre del ticket

chat

- canales por departamento
- salas por ticket
- mensajes directos internos
- presencia y lectura

notifications

- WhatsApp OTP
- correo
- push móvil
- notificación interna

attachments

- fotos, PDF, documentos, evidencia técnica
- almacenamiento MinIO configurable por BD

comments

- comentarios internos y públicos del ticket

audit

- bitácora completa
- trazabilidad por entidad y usuario

webhooks

- eventos salientes y entrantes
- firma HMAC/Bearer Token

integrations

- WhatsApp, n8n, correo, FCM, servicios externos

api-tokens

- tokens por sistema externo, área o integración

reports

- métricas, exportaciones, KPIs

dashboard

- vistas por rol y por departamento

mobile

- endpoints optimizados para Flutter
- sincronización offline básica

ai-classification

- sugerencia de clasificación, prioridad o área
- opcional y desacoplado del núcleo

7. Estructura de frontend web

Se recomienda mantener el frontend web separado del backend.

```
apps/  
  web/  
    src/  
      app/  
      modules/  
        auth/  
        dashboard/  
        tickets/  
        inventory/  
        departments/  
        templates/  
        reports/  
        chat/  
        admin/  
      components/  
      layouts/  
      hooks/  
      services/  
      store/  
      lib/  
      types/  
      pages/
```


7.1 Recomendación frontend web

- Next.js siquieres SSR, mejor SEO interno, routing maduro y mejor estructura empresarial.
- React + Vite siquieres mayor simplicidad operativa y SPA pura.

Para este caso, por enfoque interno empresarial, panel administrativo y app Flutter aparte, React + Vite es suficiente y más simple.

8. Estructura de aplicación móvil Flutter

La app Flutter debe ser independiente y orientada por rol, pero no una app distinta por rol; debe ser una sola app con navegación, features y permisos condicionados por perfil.

```
apps/  
  mobile_flutter/  
    lib/  
      core/  
        config/  
        theme/  
        network/  
        storage/  
        auth/  
        utils/  
      features/  
        auth/  
        tickets/  
        chat/  
        inventory/  
        notifications/  
        dashboard/  
        profile/  
        approvals/  
        offline_sync/  
      shared/  
        widgets/  
        models/  
        services/  
      main.dart
```

8.1 Vista por rol en Flutter

solicitante

- crear ticket rápido
- ver mis tickets
- chatear dentro del ticket
- adjuntar foto
- ver estado y resolución

tecnico_area

- bandeja de asignados
- cambiar estado
- checklist de intervención
- captura de fotos
- escaneo QR/serial
- consulta de historial del activo
- modo offline parcial

supervisor

- dashboard del área
- tickets pendientes de asignación
- aprobación o reasignación
- métricas rápidas
- chat del equipo

admin

- alertas, estado general, aprobaciones críticas, monitoreo básico

auditor

- solo lectura, reportes y trazabilidad

9. Modelo de dominio final

Las relaciones centrales del sistema deben pensarse así:

```
Usuario
├─ pertenece a uno o varios departamentos
├─ tiene uno o varios roles
├─ crea tickets
├─ atiende tickets
├─ participa en chats
└─ genera eventos de auditoría

Departamento
├─ tiene usuarios
├─ tiene activos
├─ tiene plantillas asignadas
├─ tiene flujos de ticket
└─ tiene canales de chat

Plantilla
├─ tiene versión
```

- └─ tiene campos dinámicos
- └─ define formulario/checklist/flujo
- └─ se asocia a uno o varios departamentos

Activo

- └─ pertenece a un departamento
- └─ puede tener QR/serial
- └─ tiene hoja de vida
- └─ tiene mantenimientos
- └─ se vincula a tickets

Ticket

- └─ pertenece a un departamento
- └─ usa una plantilla
- └─ puede vincular un activo
- └─ tiene solicitante
- └─ tiene técnico asignado
- └─ tiene historial
- └─ tiene mensajes/chat
- └─ tiene adjuntos
- └─ al cerrarse puede generar registro en hoja de vida

10. Diseño final de base de datos PostgreSQL

A continuación está el modelo relacional recomendado para el MVP expandible.

10.1 Tablas maestras

users

```
id UUID PK
employee_id VARCHAR(20) UNIQUE NOT NULL -- cédula
first_name VARCHAR(120)
last_name VARCHAR(120)
email VARCHAR(180) NULL
phone_whatsapp VARCHAR(30) NOT NULL
is_active BOOLEAN DEFAULT TRUE
last_login_at TIMESTAMP NULL
created_at TIMESTAMP
updated_at TIMESTAMP
```

roles

```
id UUID PK
code VARCHAR(50) UNIQUE -- admin, supervisor, tecnico_area, solicitante, auditor, i
name VARCHAR(100)
description TEXT
created_at TIMESTAMP
updated_at TIMESTAMP
```

permissions

```
id UUID PK
code VARCHAR(120) UNIQUE
name VARCHAR(150)
description TEXT
module VARCHAR(80)
created_at TIMESTAMP
updated_at TIMESTAMP
```

role_permissions

```
role_id UUID FK -> roles.id
permission_id UUID FK -> permissions.id
PRIMARY KEY (role_id, permission_id)
```

user_roles

```
id UUID PK
user_id UUID FK -> users.id
role_id UUID FK -> roles.id
department_id UUID NULL FK -> departments.id
created_at TIMESTAMP
```

departments

```
id UUID PK
code VARCHAR(50) UNIQUE
name VARCHAR(150)
description TEXT NULL
is_active BOOLEAN DEFAULT TRUE
settings_json JSONB DEFAULT '{}'
created_at TIMESTAMP
updated_at TIMESTAMP
```

user_departments

```
id UUID PK
user_id UUID FK -> users.id
department_id UUID FK -> departments.id
is_primary BOOLEAN DEFAULT FALSE
created_at TIMESTAMP
UNIQUE(user_id, department_id)
```

10.2 Autenticación y sesiones

otp_requests

```
id UUID PK
user_id UUID FK -> users.id
channel VARCHAR(30) DEFAULT 'whatsapp'
otp_code_hash VARCHAR(255)
expires_at TIMESTAMP
verified_at TIMESTAMP NULL
attempts INT DEFAULT 0
status VARCHAR(30) -- pending, verified, expired, canceled
meta_json JSONB DEFAULT '{} '
created_at TIMESTAMP
```

refresh_tokens

```
id UUID PK
user_id UUID FK -> users.id
token_hash VARCHAR(255)
device_name VARCHAR(120) NULL
device_os VARCHAR(80) NULL
ip_address INET NULL
expires_at TIMESTAMP
revoked_at TIMESTAMP NULL
created_at TIMESTAMP
```

devices

```
id UUID PK
user_id UUID FK -> users.id
platform VARCHAR(50) -- web, android, ios
app_version VARCHAR(30) NULL
push_token TEXT NULL
last_seen_at TIMESTAMP NULL
created_at TIMESTAMP
updated_at TIMESTAMP
```

10.3 Plantillas configurables

templates

```
id UUID PK
code VARCHAR(80) UNIQUE
name VARCHAR(150)
type VARCHAR(50) -- ticket_form, closure_form, checklist, workflow
version INT NOT NULL DEFAULT 1
is_active BOOLEAN DEFAULT TRUE
```

```
description TEXT NULL
schema_json JSONB NOT NULL
ui_json JSONB DEFAULT '{} '
rules_json JSONB DEFAULT '{} '
created_by UUID FK -> users.id
created_at TIMESTAMP
updated_at TIMESTAMP
```

department_templates

```
id UUID PK
department_id UUID FK -> departments.id
template_id UUID FK -> templates.id
usage_type VARCHAR(50) -- create_ticket, close_ticket, checklist, workflow
is_default BOOLEAN DEFAULT FALSE
created_at TIMESTAMP
UNIQUE(department_id, template_id, usage_type)
```

template_fields

```
id UUID PK
template_id UUID FK -> templates.id
field_key VARCHAR(100)
field_label VARCHAR(150)
field_type VARCHAR(50) -- text, textarea, select, multiselect, date, number, boolean
is_required BOOLEAN DEFAULT FALSE
sort_order INT DEFAULT 0
config_json JSONB DEFAULT '{} '
validation_json JSONB DEFAULT '{} '
created_at TIMESTAMP
updated_at TIMESTAMP
UNIQUE(template_id, field_key)
```

10.4 Inventario y hoja de vida

asset_types

```
id UUID PK
code VARCHAR(80) UNIQUE
name VARCHAR(120)
description TEXT NULL
schema_json JSONB DEFAULT '{} '
created_at TIMESTAMP
updated_at TIMESTAMP
```

assets

```
id UUID PK
department_id UUID FK -> departments.id
asset_type_id UUID FK -> asset_types.id
internal_code VARCHAR(80) NULL
serial_number VARCHAR(120) NULL
qr_code VARCHAR(120) NULL
name VARCHAR(180)
brand VARCHAR(120) NULL
model VARCHAR(120) NULL
location VARCHAR(180) NULL
status VARCHAR(40) -- active, maintenance, inactive, retired
purchase_date DATE NULL
warranty_end DATE NULL
metadata_json JSONB DEFAULT '{} '
is_active BOOLEAN DEFAULT TRUE
created_at TIMESTAMP
updated_at TIMESTAMP
UNIQUE(serial_number)
UNIQUE(qr_code)
```

asset_lifecycle_entries

```
id UUID PK
asset_id UUID FK -> assets.id
entry_type VARCHAR(50) -- install, preventive, corrective, calibration, incident, cl
source_type VARCHAR(50) -- manual, ticket, import, api
source_id UUID NULL
summary TEXT NOT NULL
detail_json JSONB DEFAULT '{} '
performed_by UUID NULL FK -> users.id
performed_at TIMESTAMP
created_at TIMESTAMP
```

asset_documents

```
id UUID PK
asset_id UUID FK -> assets.id
attachment_id UUID FK -> attachments.id
document_type VARCHAR(50) -- manual, warranty, calibration, report, photo
created_at TIMESTAMP
```

10.5 Tickets

ticket_statuses

```
id UUID PK
code VARCHAR(50) UNIQUE -- open, triage, assigned, in_progress, pending_parts, resc
name VARCHAR(120)
category VARCHAR(50) -- active, final
sort_order INT DEFAULT 0
is_default BOOLEAN DEFAULT FALSE
is_closed BOOLEAN DEFAULT FALSE
created_at TIMESTAMP
```

ticket_priorities

```
id UUID PK
code VARCHAR(50) UNIQUE -- low, medium, high, critical
name VARCHAR(100)
response_minutes INT NULL
resolution_minutes INT NULL
created_at TIMESTAMP
```

tickets

```
id UUID PK
ticket_number BIGSERIAL UNIQUE
department_id UUID FK -> departments.id
template_id UUID NULL FK -> templates.id
requester_id UUID FK -> users.id
assigned_to UUID NULL FK -> users.id
supervisor_id UUID NULL FK -> users.id
asset_id UUID NULL FK -> assets.id
status_id UUID FK -> ticket_statuses.id
priority_id UUID FK -> ticket_priorities.id
subject VARCHAR(200)
description TEXT
channel VARCHAR(30) DEFAULT 'web' -- web, mobile, api
classification_source VARCHAR(30) DEFAULT 'manual' -- manual, ai, rule
reported_at TIMESTAMP DEFAULT NOW()
assigned_at TIMESTAMP NULL
first_response_at TIMESTAMP NULL
resolved_at TIMESTAMP NULL
closed_at TIMESTAMP NULL
sla_due_at TIMESTAMP NULL
closure_summary TEXT NULL
custom_data_json JSONB DEFAULT '{}'
created_at TIMESTAMP
updated_at TIMESTAMP
```


ticket_events

```
id UUID PK
ticket_id UUID FK -> tickets.id
event_type VARCHAR(80) -- created, assigned, status_changed, commented, attachment_a
actor_user_id UUID NULL FK -> users.id
old_value_json JSONB DEFAULT '{} '
new_value_json JSONB DEFAULT '{} '
notes TEXT NULL
created_at TIMESTAMP
```

ticket_comments

```
id UUID PK
ticket_id UUID FK -> tickets.id
user_id UUID FK -> users.id
comment_type VARCHAR(30) -- public, internal
content TEXT
created_at TIMESTAMP
updated_at TIMESTAMP
```

ticket_attachments

```
id UUID PK
ticket_id UUID FK -> tickets.id
attachment_id UUID FK -> attachments.id
attachment_role VARCHAR(50) -- evidence_before, evidence_after, report, general
created_by UUID FK -> users.id
created_at TIMESTAMP
```

ticket_form_values

```
id UUID PK
ticket_id UUID FK -> tickets.id
template_field_id UUID FK -> template_fields.id
value_json JSONB NOT NULL
created_at TIMESTAMP
updated_at TIMESTAMP
UNIQUE(ticket_id, template_field_id)
```

ticket_assignments

```
id UUID PK
ticket_id UUID FK -> tickets.id
assigned_to UUID FK -> users.id
assigned_by UUID FK -> users.id
assignment_type VARCHAR(30) -- manual, automatic, escalation
assigned_at TIMESTAMP
```

```
released_at TIMESTAMP NULL
notes TEXT NULL
```

10.6 Workflows y SLA

workflow_definitions

```
id UUID PK
department_id UUID FK -> departments.id
name VARCHAR(120)
is_active BOOLEAN DEFAULT TRUE
config_json JSONB NOT NULL
created_at TIMESTAMP
updated_at TIMESTAMP
```

workflow_transitions

```
id UUID PK
workflow_id UUID FK -> workflow_definitions.id
from_status_id UUID FK -> ticket_statuses.id
to_status_id UUID FK -> ticket_statuses.id
requires_comment BOOLEAN DEFAULT FALSE
requires_resolution BOOLEAN DEFAULT FALSE
requires_checklist BOOLEAN DEFAULT FALSE
requires_supervisor_approval BOOLEAN DEFAULT FALSE
config_json JSONB DEFAULT '{} '
created_at TIMESTAMP
```

sla_policies

```
id UUID PK
department_id UUID FK -> departments.id
name VARCHAR(120)
priority_id UUID FK -> ticket_priorities.id
response_minutes INT
resolution_minutes INT
business_hours_only BOOLEAN DEFAULT TRUE
config_json JSONB DEFAULT '{} '
is_active BOOLEAN DEFAULT TRUE
created_at TIMESTAMP
updated_at TIMESTAMP
```

10.7 Chat empresarial

chat_channels

```
id UUID PK
department_id UUID NULL FK -> departments.id
name VARCHAR(120)
channel_type VARCHAR(30) -- department, direct, ticket, announcement
is_private BOOLEAN DEFAULT FALSE
created_by UUID FK -> users.id
created_at TIMESTAMP
updated_at TIMESTAMP
```

chat_channel_members

```
id UUID PK
channel_id UUID FK -> chat_channels.id
user_id UUID FK -> users.id
role_in_channel VARCHAR(30) DEFAULT 'member'
joined_at TIMESTAMP
last_read_message_id UUID NULL
UNIQUE(channel_id, user_id)
```

chat_messages

```
id UUID PK
channel_id UUID FK -> chat_channels.id
sender_user_id UUID FK -> users.id
message_type VARCHAR(30) DEFAULT 'text' -- text, file, system
body TEXT NULL
meta_json JSONB DEFAULT '{} '
created_at TIMESTAMP
updated_at TIMESTAMP NULL
deleted_at TIMESTAMP NULL
```

chat_message_attachments

```
id UUID PK
message_id UUID FK -> chat_messages.id
attachment_id UUID FK -> attachments.id
created_at TIMESTAMP
```

10.8 Adjuntos y almacenamiento

storage_providers

```
id UUID PK
code VARCHAR(50) UNIQUE -- local, minio, s3
name VARCHAR(100)
is_default BOOLEAN DEFAULT FALSE
config_json JSONB NOT NULL
is_active BOOLEAN DEFAULT TRUE
created_at TIMESTAMP
updated_at TIMESTAMP
```

attachments

```
id UUID PK
storage_provider_id UUID FK -> storage_providers.id
bucket_name VARCHAR(120)
object_key VARCHAR(255)
original_name VARCHAR(255)
mime_type VARCHAR(120)
size_bytes BIGINT
checksum_sha256 VARCHAR(255) NULL
uploaded_by UUID NULL FK -> users.id
visibility VARCHAR(30) DEFAULT 'private'
meta_json JSONB DEFAULT '{} '
created_at TIMESTAMP
```

Este diseño permite que MinIO sea configurable por base de datos, e incluso cambiar luego a S3 sin romper la capa de negocio.

10.9 Notificaciones e integraciones

notification_templates

```
id UUID PK
code VARCHAR(80) UNIQUE
name VARCHAR(120)
channel VARCHAR(30) -- whatsapp, email, push, in_app
subject_template TEXT NULL
body_template TEXT NOT NULL
variables_json JSONB DEFAULT '[]'
is_active BOOLEAN DEFAULT TRUE
created_at TIMESTAMP
updated_at TIMESTAMP
```

notification_logs

```
id UUID PK
user_id UUID NULL FK -> users.id
channel VARCHAR(30)
recipient VARCHAR(180)
template_id UUID NULL FK -> notification_templates.id
status VARCHAR(30) -- queued, sent, delivered, failed
provider VARCHAR(50)
provider_message_id VARCHAR(120) NULL
payload_json JSONB DEFAULT '{} '
response_json JSONB DEFAULT '{} '
created_at TIMESTAMP
sent_at TIMESTAMP NULL
```

api_tokens

```
id UUID PK
name VARCHAR(120)
owner_type VARCHAR(30) -- system, department, user
owner_id UUID NULL
token_hash VARCHAR(255)
scopes_json JSONB DEFAULT '[]'
last_used_at TIMESTAMP NULL
expires_at TIMESTAMP NULL
is_active BOOLEAN DEFAULT TRUE
created_at TIMESTAMP
```

webhook_endpoints

```
id UUID PK
name VARCHAR(120)
direction VARCHAR(20) -- inbound, outbound
url TEXT NULL
bearer_token_hash VARCHAR(255) NULL
hmac_secret_hash VARCHAR(255) NULL
events_json JSONB DEFAULT '[]'
is_active BOOLEAN DEFAULT TRUE
created_at TIMESTAMP
updated_at TIMESTAMP
```

webhook_deliveries

```
id UUID PK
webhook_id UUID FK -> webhook_endpoints.id
event_name VARCHAR(120)
entity_type VARCHAR(80)
entity_id UUID
request_json JSONB
response_status INT NULL
```

```
response_body TEXT NULL
attempt_count INT DEFAULT 0
status VARCHAR(30) -- pending, success, failed
created_at TIMESTAMP
processed_at TIMESTAMP NULL
```

10.10 Auditoría

audit_logs

```
id UUID PK
actor_user_id UUID NULL FK -> users.id
entity_type VARCHAR(80)
entity_id UUID NULL
action VARCHAR(80)
ip_address INET NULL
user_agent TEXT NULL
old_values_json JSONB DEFAULT '{}'
new_values_json JSONB DEFAULT '{}'
meta_json JSONB DEFAULT '{}'
created_at TIMESTAMP
```

11. Reglas clave de negocio

11.1 Creación de ticket

1. El usuario inicia sesión con cédula + OTP WhatsApp.
2. Elige departamento.
3. Elige tipo de solicitud o plantilla.
4. Busca activo por serial, QR o nombre, si aplica.
5. El sistema carga campos dinámicos según plantilla del departamento.
6. Se crea ticket con trazabilidad completa.
7. Se dispara evento de notificación y webhook.

11.2 Asignación

- Puede ser manual por supervisor.
- Puede ser automática por reglas: carga, especialidad, horario, criticidad.
- Todo cambio queda auditado.

11.3 Cierre técnico

Al cerrar un ticket:

- se exige resolución,
- puede exigirse checklist,
- puede exigirse evidencia fotográfica,
- se guarda comentario de cierre,
- si existe activo vinculado, se crea automáticamente un `asset_lifecycle_entry`.

11.4 Plantillas por departamento

Ejemplo:

- Biomédico: fabricante, modelo, serie, servicio clínico, criticidad clínica, riesgo paciente.
- Sistemas: hostname, serial PC, usuario afectado, sede, software implicado.
- Aire acondicionado: BTU, ubicación, compresor, presión, tipo de falla.
- Mantenimiento de precisión: tolerancia, calibración, desviación, instrumento asociado.

12. Estructura recomendada del repositorio

```
company-helpdesk/
├── apps/
│   ├── api/           # NestJS backend
│   ├── web/           # React/Vite frontend web
│   └── mobile_flutter/ # Flutter app nativa
├── packages/
│   ├── shared-types/  # DTOs/contratos si luego quieres compartir esquemas
│   ├── eslint-config/
│   └── tsconfig/
├── infrastructure/
│   ├── docker/
│   │   ├── nginx/
│   │   ├── postgres/
│   │   ├── redis/
│   │   ├── minio/
│   │   └── n8n/
│   ├── compose/
│   │   ├── docker-compose.dev.yml
│   │   └── docker-compose.prod.yml
│   └── scripts/
├── docs/
│   ├── architecture/
│   ├── api/
│   ├── database/
│   └── flows/
├── .env.example
└── README.md
```

13. Infraestructura open source recomendada

Para cumplir tu objetivo de usar lo máximo open source posible, esta sería la pila ideal:

Capa	Tecnología
Backend API	NestJS
ORM	Prisma o TypeORM
Base de datos	PostgreSQL
Cache / cola / rate limit	Redis
Jobs	BullMQ
Archivos	MinIO
Reverse proxy	Nginx
Orquestación simple	Docker Compose
Panel despliegue	EasyPanel
Automatización	n8n
Push móvil	Firebase Cloud Messaging
Observabilidad	Grafana + Loki + Prometheus
Errores	Sentry self-hosted opcional / GlitchTip
Búsqueda futura	OpenSearch

13.1 Sobre autenticación open source

Como tu login real depende de OTP por WhatsApp, puedes mantener autenticación propia en NestJS. Si luego quieres SSO corporativo o federación, puedes evaluar Keycloak como capa open source adicional.

14. Modelo de permisos recomendado

Se recomienda combinar:

- RBAC: rol base.
- Scope por departamento.
- Permisos granulares por módulo.
- Restricción por canal si hace falta.

Ejemplo:

Permiso	Admin	Supervisor	Técnico	Solicitante	Auditor
tickets.create	Sí	Sí	Sí	Sí	No
tickets.assign	Sí	Sí	No	No	No

Permiso	Admin	Supervisor	Técnico	Solicitante	Auditor
tickets.close	Sí	Sí	Sí	No	No
inventory.manage	Sí	Sí	Según área	No	No
templates.manage	Sí	Opcional	No	No	No
reports.view	Sí	Sí	Parcial	Propios	Sí
audit.view	Sí	Parcial	No	No	Sí
webhooks.manage	Sí	No	No	No	No

15. API recomendada

15.1 REST principal

Rutas ejemplo:

```

POST    /auth/request-otp
POST    /auth/verify-otp
POST    /auth/refresh
POST    /auth/logout

GET     /me
GET     /dashboard/summary

GET     /departments
GET     /departments/:id/templates

GET     /tickets
POST    /tickets
GET     /tickets/:id
PATCH  /tickets/:id
POST    /tickets/:id/assign
POST    /tickets/:id/change-status
POST    /tickets/:id/close
GET     /tickets/:id/timeline

GET     /assets
POST    /assets
GET     /assets/:id
GET     /assets/:id/lifecycle

GET     /templates
POST    /templates
PATCH  /templates/:id

POST    /attachments/presign
POST    /webhooks/tickets

```

15.2 WebSocket

Namespaces sugeridos:

```
/ws/chat  
/ws/notifications  
/ws/tickets
```

Eventos:

- chat.message.sent
- chat.message.read
- ticket.created
- ticket.assigned
- ticket.status.changed
- ticket.closed
- notification.push

16. Modelo final de datos simplificado

```
Department 1---N Users  
Department 1---N Assets  
Department 1---N Tickets  
Department N---N Templates  
  
User N---N Roles  
User 1---N Tickets(requester)  
User 1---N Tickets(assigned_to)  
  
Asset 1---N Ticket  
Asset 1---N AssetLifecycleEntry  
  
Template 1---N TemplateFields  
Template 1---N Tickets  
  
Ticket 1---N TicketEvents  
Ticket 1---N TicketComments  
Ticket 1---N TicketAttachments  
Ticket 1---N TicketFormValues  
  
ChatChannel 1---N ChatMessages  
ChatChannel N---N Users
```

17. Estrategia de fases refinada

Fase 1 — MVP sólido

- Autenticación OTP WhatsApp
- Usuarios, roles, departamentos
- Tickets básicos
- Inventario simple
- Activo vinculado al ticket
- Cierre con escritura automática en hoja de vida
- Chat por ticket y por departamento
- Web administrativa
- Flutter con solicitante y técnico
- MinIO configurable por base de datos

Fase 2 — Operación avanzada

- Plantillas dinámicas por DB
- Checklist de cierre
- Asignación automática
- SLA y escalamiento
- Webhooks n8n
- Dashboard por rol
- push notifications
- offline básico Flutter

Fase 3 — Plataforma expandible

- Nuevos departamentos sin desarrollo fuerte
- IA para clasificación
- búsqueda OpenSearch
- reportes avanzados
- auditoría enriquecida
- multiempresa si luego deseas volverlo SaaS interno/externo

18. Recomendaciones técnicas finales

18.1 Decisiones recomendadas

- Backend: NestJS.
- ORM: Prisma si quieres velocidad de desarrollo y esquema claro; TypeORM si prefieres mayor cercanía al ecosistema clásico Nest.
- Web: React + Vite.
- Móvil: Flutter nativo independiente.
- DB: PostgreSQL.
- Archivos: MinIO con proveedor configurable desde DB.
- Tiempo real: Socket.IO.
- Jobs: BullMQ.

18.2 No recomendado para esta etapa

- Microservicios desde el inicio.
- Duplicar lógica de negocio en web y móvil.
- Hardcodear departamentos o formularios en frontend.
- Acoplar MinIO directamente en código sin capa `storage_providers`.
- Crear una app separada por rol; mejor una sola app Flutter con capacidades por permiso.

19. Propuesta de modelo final resumido

El sistema final quedaría así:

- Un backend NestJS modular, central y escalable.
- Una base de datos PostgreSQL con departamentos, plantillas, tickets, inventario, chat y auditoría.
- Una app web para gestión administrativa y operativa.
- Una app móvil Flutter nativa para operación en campo y seguimiento por rol.
- Plantillas y formularios controlados por base de datos, asociados a departamentos.
- Inventario con hoja de vida automática, alimentada por el cierre de tickets.
- Infraestructura mayormente open source, desplegable en Docker Compose + EasyPanel.

20. Próximo nivel recomendado

Los tres entregables más útiles para continuar serían:

1. Diagrama ER completo de la base de datos.
2. Estructura real de carpetas NestJS con módulos, controladores, servicios y entidades.
3. SQL inicial del esquema PostgreSQL para arrancar el proyecto.